

Theory Of Computation

① ~~Basic concepts~~ FA, NFA, DFA, ENFA, RE, DATE

Equivalence between FA and RE, Minimization,

Pumping Lemma for RE.

Finite Automata

Machine

5 tuple : $\{Q, \Sigma, q_0, F, \delta\}$

Transition Diagram

Graph

Function

Automaton = An abstract computing device.

Automata theory

theoretical branch of computer science and mathematics

Finite Automata

- Graphical (Trans. Diagram)
- Tabular (Trans. Table)
- Mathematical (Trans func.)

5 tuple :

$M = (Q, \Sigma, \delta, q_0, F)$

$\delta : Q \times \Sigma \rightarrow Q$

states

Alphabets

Initial state

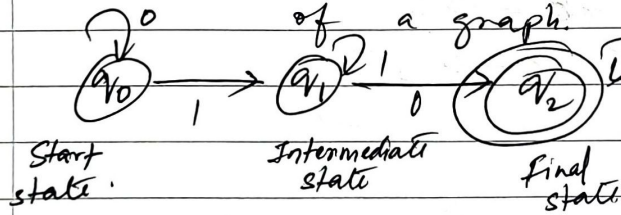
Final state

Transition function

Transition Diagram

= Directed graph associated with restriction of a graph.

eg:



inputs : $\{0, 1\}$

Transition table

δ	0	1
q_0	q_0	q_1
q_1	q_2	q_1
q_2	q_2	q_2

Transition function

$\delta(\text{current state}, \text{current input symbol}) = \text{next state}$

DFA: Types of finite Automate.

DATE

Finite Automata without
o/p

Finite Automata with
o/p

DFA

NFA

ENFA

Moore
machine

Mealy
machine

DFA: Single-path-determination length.

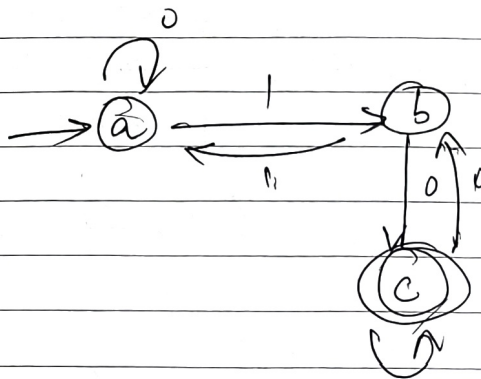
no. of. state = |min-string| + 1

Example s:

① DFA = $Q = \{a, b, c\}$
 $\Sigma = \{0, 1\}$

	0	1
a	a	b
b	c	a
c	b	c

Trans. table



DFA $\Rightarrow \delta: Q \times \Sigma \rightarrow Q$.

NFA $\Rightarrow \delta: Q \times \Sigma \rightarrow 2^Q$

ENFA $\Rightarrow \delta: Q \times \Sigma \cup \epsilon \rightarrow 2^Q$.

① Design DFA: $\Sigma = \{0, 1\}$ accepts even no. of 0's and even no. of 1's

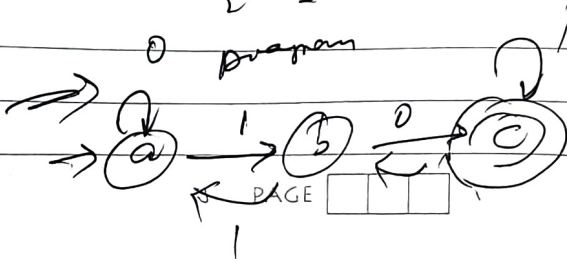
no. of. states = |min length| + 1.

$F = 2 + 1 = 3$.

① Let DFA be $Q = \{a, b, c\}$ $q_0 = \{a\}$
 $\Sigma = \{0, 1\}$ $F = \{c\}$

classmate

	0	1
a	a	b
b	c	a
c	b	c



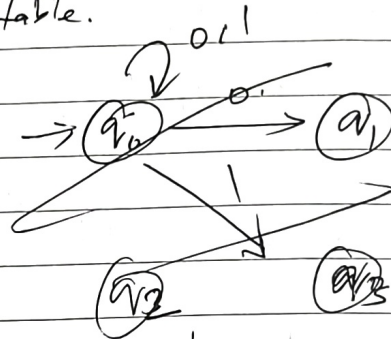
PAGE

→ ~~NFA~~: $\delta: Q \times \Sigma \rightarrow 2^Q$ → Multiple Next states.
 $\delta: Q \times \Sigma \rightarrow 2^Q$ DATE

Every NFA is not DFA

① Design NFA for the given transition table.

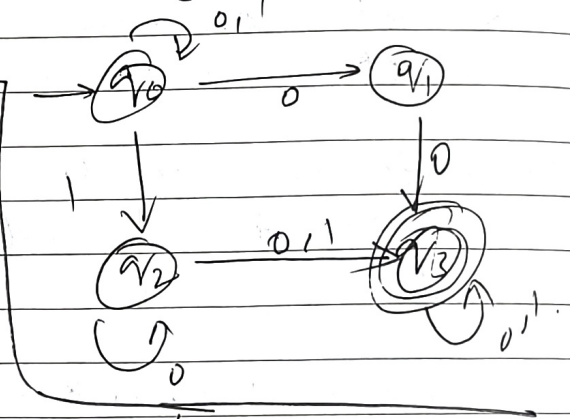
state	0	1
q_0	q_0, q_1	q_0, q_2
q_1	q_3	ϵ
q_2	q_2, q_3	q_3
q_3	q_3	q_3



② ~~NFA~~ → DFA:

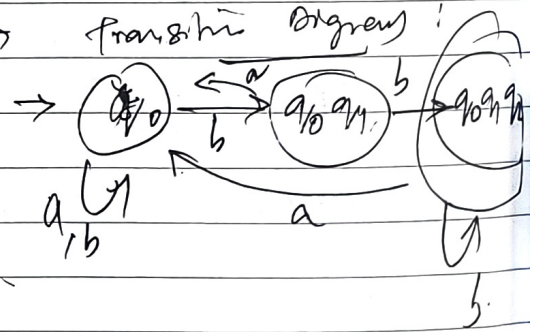
δ	a	b
q_0	q_0	$\{q_0, q_1\}$
q_1	-	q_2
q_2	-	-

→ NFA table



③ constructing to DFA → transition diagram:

δ	a	b
q_0	q_0	$\{q_0, q_1\}$
$\{q_0, q_1\}$	q_0	$\{q_0, q_1, q_2\}$
$\{q_0, q_1, q_2\}$	q_0	$\{q_0, q_1, q_2\}$



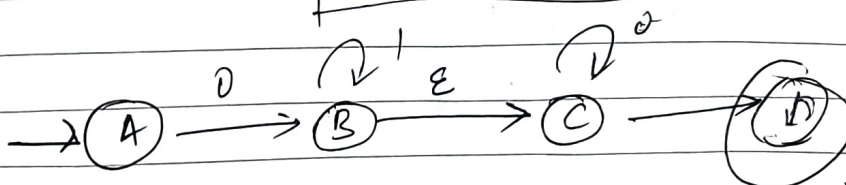
→ NFA with ϵ :

ϵ = Empty symbol (Epsilon)

5 tuple $\{Q, \Sigma, q_0, F, \delta\}$

where:

$$\delta: Q \times \Sigma \cup \epsilon \rightarrow 2^Q$$



classmate NOTE: B on seeing nothing, goes to 'C'. Every state on ϵ goes to itself. PAGE

* RE, DFA minimization, RE $\rightarrow$ Finite Automata.

* Regular Expressions :-

Sequence of pattern that defines a string.

- $\rightarrow$ Union (U)
- $\rightarrow$ Kleen (*)
- $\rightarrow$ Plus (+) 0 or more occurrence
- $\rightarrow$ Optional (?)

(P) Write RE for language accepting all strings containing any no. of a's and b's. Also construct FA Diagram

sol $L = \{ \epsilon, a, b, aa, bb, ab, ba, abb, baa, \dots \}$

RE: $(a + b)^*$ $\rightarrow$ (a) (b)

x^+ = one/more occurrences of x.

x^* = zero/more occurrences of x.

(P) Write RE for the language accepting all strings starting with 1 and ending with 0, over $\Sigma = \{0, 1\}$

$L = \{ 10, 100, 110, 1000, 1110, 1010, \dots \}$

RE = $1(0+1)^*0$

(L: Language)

(P) Write RE for L over $\Sigma = \{0\}$ having even length of a string.

$L = \{ 00, 0000, 000000, \dots \}$

RE = $(00)^*$

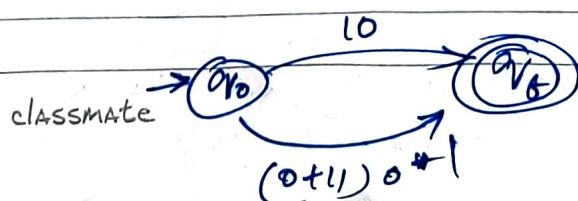
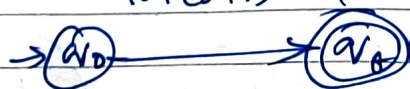
Conversion of RE to FA :-

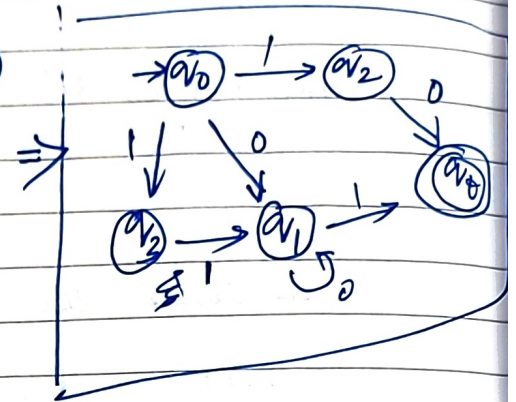
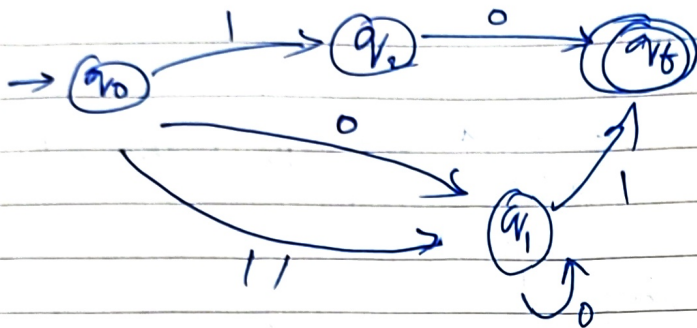
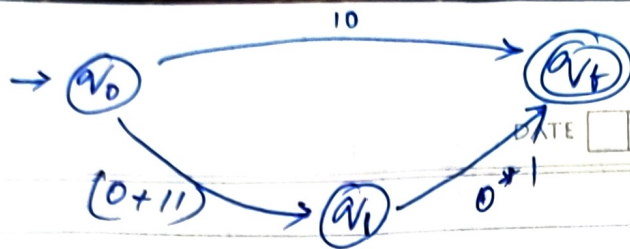
Step 1: Design transition dig for given RE, using NFA with ϵ moves.

Step-2: Convert NFA with ϵ to NFA without ϵ .

Step-3: Convert the resultant NFA to DFA.

(P) Design a FA for the given RE: $10 + (0+11)0^*$





Module - 2

Context Free Grammar : (4-tuple)

$$G = \{N, T, P, S\}$$

used to generate all possible strings in a finite language.

Non-terminal symbol
(caps)
A, B, C, D

Terminal symbol
(non-caps)
a, b, c, d

Production rule
used to replace non-terminals on left side of

start symbol.

production with terminal & non-terminal symbols.

Q. Construct CFG for the language having no. of a's over the set $\Sigma = \{a\}$. (10)

$$A = \{N, T, P, S\}$$

$$L = \{ \epsilon, a, aa, aaa, aaaa, \dots \}$$

Step 1: Derive i/p string: aaaaa

Production rule:

$$S \rightarrow aS$$

$$S \rightarrow \epsilon$$

Production rule:

start

S

aS

a(aS)

a(a(aS))

a(a(a(aS)))

$$S \rightarrow \epsilon$$

$$S \rightarrow aS$$

$$S \rightarrow aS$$

$$S \rightarrow aS$$

$$S \rightarrow aS$$

$$S \rightarrow \epsilon$$

S

aS

a(aS)

a(a(aS))

a(a(a(aS)))

aaaaa → i/p string is derived.

① Construct a CFG for language:

$$L = \{ w c w^R \mid \text{where } w \in (a, b)^* \}$$

Ex $\downarrow$ reverse of a string.

$$L = \{ aca, abcb a, aabcb aa, \dots \}$$

Grammar could be:

$$\left. \begin{array}{l} S \rightarrow asa \\ S \rightarrow bsb \\ S \rightarrow c \end{array} \right\} \text{production rules.}$$

Lets derive the string aabcb aa.

$$\begin{array}{lcl} \text{start} & & aSa \\ S \rightarrow asa & a(asa)a. & \\ S \rightarrow bsb & aa(bsb)aa & \\ S \rightarrow c & aab(c)baa & \\ \Rightarrow & \boxed{aabcb aa} & \text{i/p string is derived.} \end{array}$$

$\therefore$ Hence, CFG is constructed.

② Construct CFG for language: $L = a^n b^{2n}$ where $n \geq 1$.

Ex

$$L = \{ abb, aabbbb, aaabbbbbbb, \dots \}$$

Production rule:

$$\boxed{\begin{array}{l} S \rightarrow aSbb \\ S \rightarrow \epsilon \end{array}}$$

Lets,

$\Rightarrow$ Derive i/p string aaabbbbbbb:

Derivation:

$$\begin{array}{lcl} \text{start} & & S \\ S \rightarrow aSbb & aSbb & \\ S \rightarrow aSbb & a(aSbb)bb & \\ S \rightarrow aSbb & aa(aSbb)bbbb & \\ S \rightarrow \epsilon & \boxed{aaabbbbbbb} & \text{i/p string is derived.} \end{array}$$

$\therefore$ CFG is constructed.

⇒

PushDown Automata

Finite state machine + stack

DATE

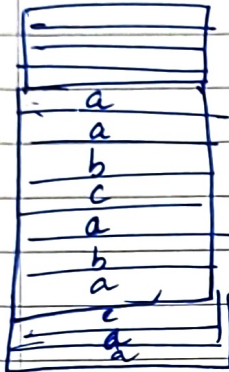
□ □ □ □ □ □ □ □

- Finite state machine
- A control unit
- stack of infinite size.

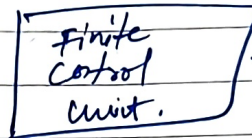
7-tuples : $(Q, \Sigma, \Gamma, \delta, q_0, I, F)$

Stack symbol

initial stack top symbol.

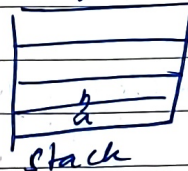


take I/P



Accept/Reject

push/pop



~~stack of infinite size~~
input tape

"no size"

!!! * must look into PDA

check problems.

Module-3

CNF = Chomsky Normal Form

CFG to CNF :-

* Decompose or breakdown the grammar.

A start symbol generates ϵ

eg: $A \rightarrow \epsilon$

A non-terminal generating two non-terminal.

eg: $S \rightarrow AB$

A nonterminal generating a single terminal.

eg: $S \rightarrow a$

① Convert the CFG to CNF:

$S \rightarrow a | aA | B$

$A \rightarrow aB | \epsilon$

$B \rightarrow Aa | b$

Sol Some of the production rules are not in CNF.

Step: 1 Add new production to it.

$S_1 \rightarrow S$ // new production

CNF	GNF
* Chomsky Normal Form	Greibach Normal form
$S \rightarrow E$ $A \rightarrow a$ $A \rightarrow AB$	$S \rightarrow E$ $A \rightarrow a$ $A \rightarrow aABE$

Learn more!!!

Ambiguity in Grammar :-

- * If $n(\text{parsetrees}) > 1$
 then Grammar G is Ambiguous.
- if no. of parse trees > 1
 then Grammar G is Ambiguous.

⑧ Consider the production rule; check Ambiguity.

$E \rightarrow I$
 $E \rightarrow E + E$
 $E \rightarrow E * E$
 $E \rightarrow (E)$
 $I \rightarrow \epsilon | 0 | 1 | \dots$

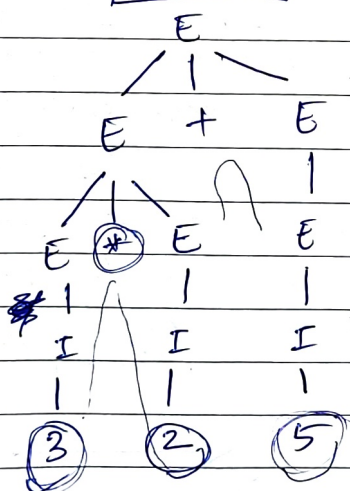
So Take any string:

~~consider~~
Step-1 Consider string "3*2+5".

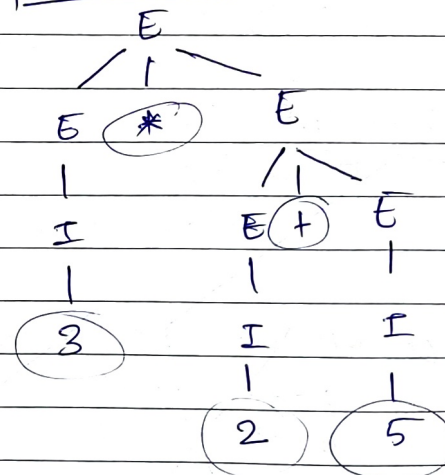
3*2+5

Step-2:

Tree-1



Tree-2



$\therefore 3 * 2 + 5$
 is derived ✓

$\therefore 3 * 2 + 5$

$\therefore$ From the given grammar, two parse trees can be formed. Hence, Grammar G is Ambiguous.

Parse-trees.

- * Root node is always a node indicating start symbols
- * The derivation is read from left to right.
- * The Leaf node are always Terminal nodes.
- * Interior nodes are always Non-Terminals.

Example

Production rule

$$E \rightarrow E + E$$

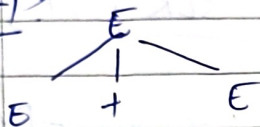
$$E \rightarrow E * E$$

$$E \rightarrow a | b | c$$

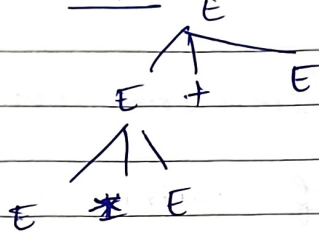
Input-1

$$a * b + c$$

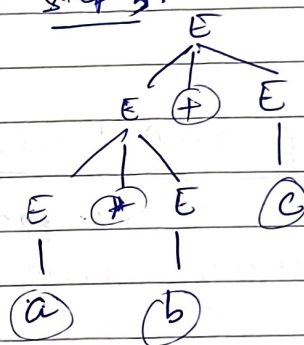
Step-1



Step-2



..... Step-3

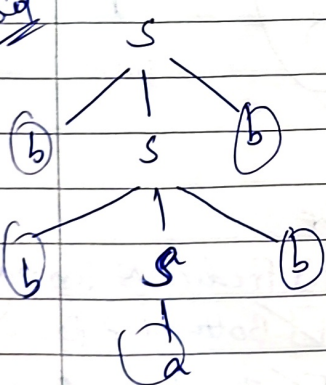


Input string is derived:

$$a * b + c$$

Q. Draw a derivation tree for the string "bbabb",
from CFG given by: $S \rightarrow bsb | a | b$.

Sol



Derivation tree

"bbabb" is derived

Q.

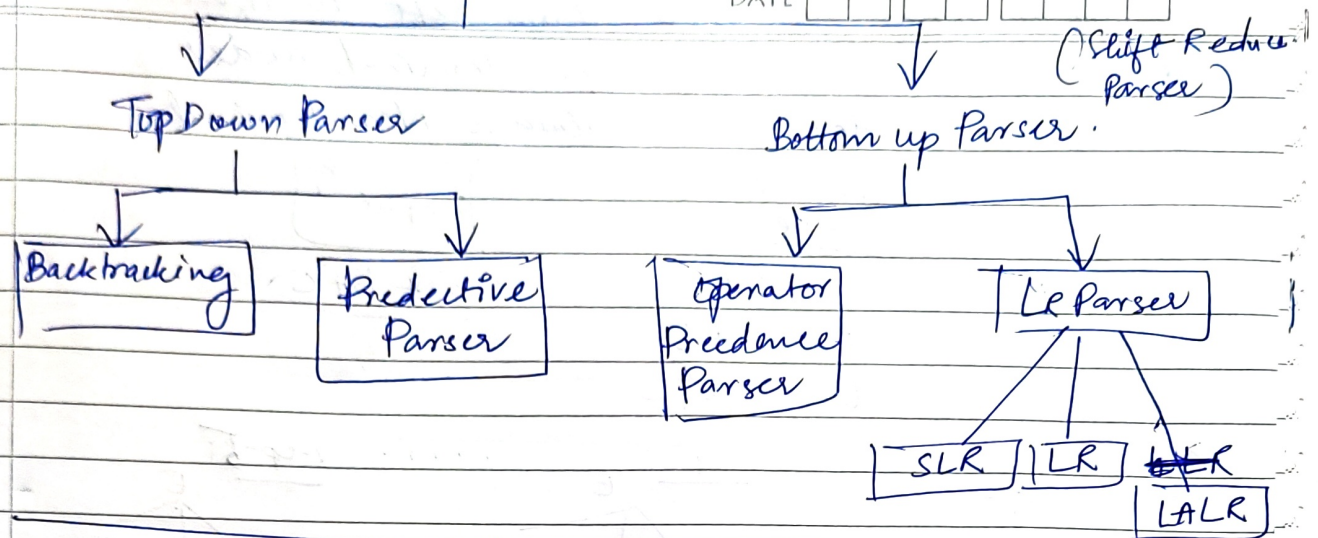
Parser

→ Parsing

→ ~~Analysing~~ Process of analyzing a string of symbols (like program or sentence) to determine its grammatical structure according to a set of defined rules (grammar).

Types of Parser

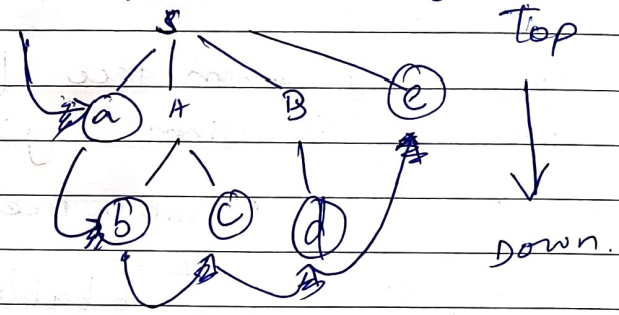
DATE



Top-down Parser : (Start from root + Left most Derivation)

→ The process of construction of parse tree starting from root & proceed to children.
 → i.e., starting from start symbol to reaching the i/p string.

$S \rightarrow aABe$
 $A \rightarrow bc$
 $B \rightarrow d$
 $w \rightarrow abcde$



Top down

Recursive Descent

Backtracking

Non-backtracking

~~Brute force technique~~

Predictive Parser

LL Parser

- ☒ Predictive Parsing
- ☒ BottomUP Parsing
- ☒ Topdown Parsing
- ☐ SLR
- ☐ CLR
- ☐ LALR
- ☐ YACC
- ☐ CFG → CNF
- ☐ CFG → GNF

Module-3

Bottom Parsing: (Rightmost Derivations + reverse)

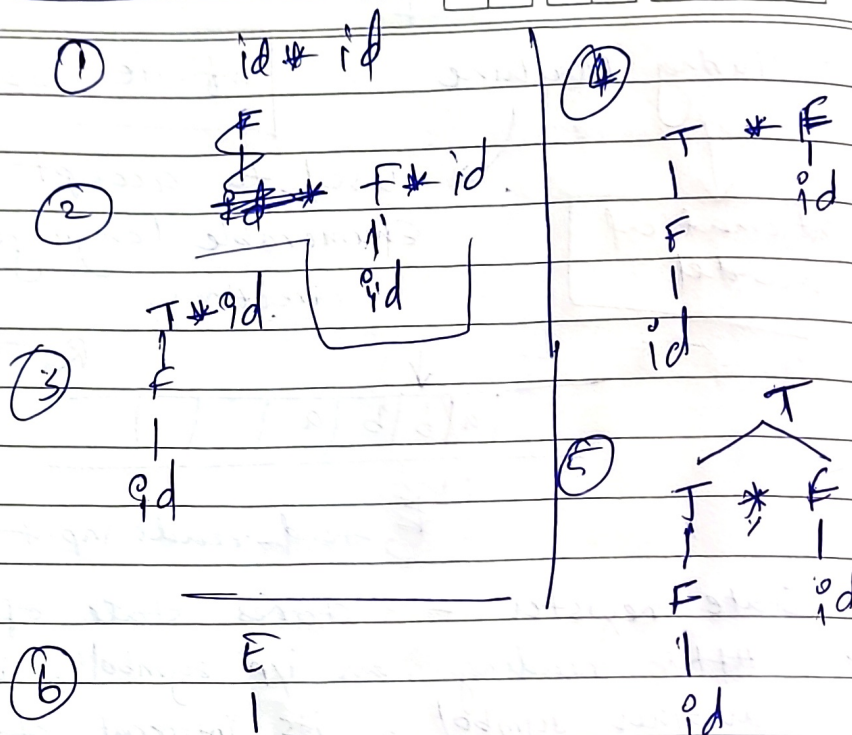
- ① Production rule
- $$E \rightarrow E + T \mid T$$
- $$T \rightarrow T * F \mid F$$
- $$F \rightarrow id$$

Derive the given string

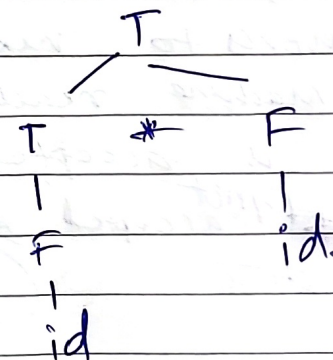
Derive the string $id * id$.

DATE

--	--	--	--	--	--	--	--



Bottom-up parsing



⇒ Predictive Parsing ⇒ LL(1) parsing

* Used to predict which of the production rules to follow ^{or use} from the given data with the help of LL(1) parsing table & the i/p strings.

②

✗

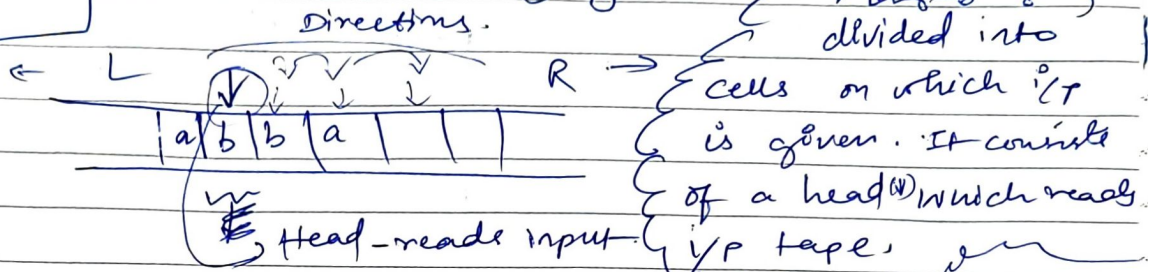
MODULE - 4

DATE

⇒ Turing Machine - infinite size tape

mathematical model

used to accept recursive Enumerable language



- * State register - stores state of T.M.
- * After reading an i/p symbol, it is replaced with another symbol, its internal state is changed, and it moves to next cell - either right/left.
- * If Turing machine reaches the final state, the i/p string is accepted; otherwise rejected.
- * If ~~not~~ ~~not~~ accepted, then it's in "Halt" - non-final state.

States

q_0, q_1, q_2

7 tuple

$(Q, \Sigma, \Gamma, \delta, q_0, F)$

Tape Alphabet

i/p alphabet

Blank symbol (Δ)

Replaced with other symbol

a, b, b, Δ, Δ

x, y, y

$\delta: Q \times \Sigma \rightarrow Q \times \Sigma \times \{\text{leftshift, Rightshift}\}$

Basic Model

- * i/p tape has ∞ no. of cells.
- * each cell contains one i/p symbol.
- * the empty tape is filled by blank characters.

$\dots a, b, c, \Delta, \Delta, \Delta, \dots$ i/p tape

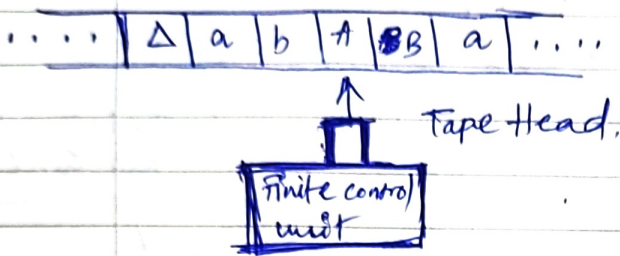
Finite control unit

- * Tape head responsible for reading the current i/p symbol. Tape head can be read from left to right.

* A finite set of states through which machine has to undergo

DATE

* Finite set of symbols called external symbols which are used in building the logic of TM.



Δ - Blank symbol
- used to fill empty gaps in infinite tapes.

⇒ Turing Thesis: TM as a computational model.

* TM can simulate any algorithmic logic. It consists of

- tape
- Head
- State register
- A finite table

⇒ TM as a recognizer:

As a recognizer, a TM checks if a string belongs to a particular language. It:
Accepts a string if it ~~ends~~ halts in a non accepting state / loops infinitely.

⇒ ~~Turing Machine~~ Other models of ~~Finite~~ TM

models	feature	Rever increase
	Linear.	

⇒ Linear Bounded Automaton (LBA) Non-deterministic Turing Machine

* Special kind of TM with restricted tape size.
* i.e., limited tape size
* ~~Non-deterministic~~

$L_{LBA} = (Q, \Sigma, \gamma, \delta, q_0, q_{accept}, q_{reject})$

~~same as ordinary TM but additional~~
classmate rule: ~~the tape head not allowed to~~
~~move beyond area of tape.~~

* LBA is ~~same size~~ as some as a standard Turing Machine, but with one main key constraint—

DATE

"The read/write head is not allowed to move outside the tape ~~write~~ area of the tape that contains original input."

* Given input size is the tape size.

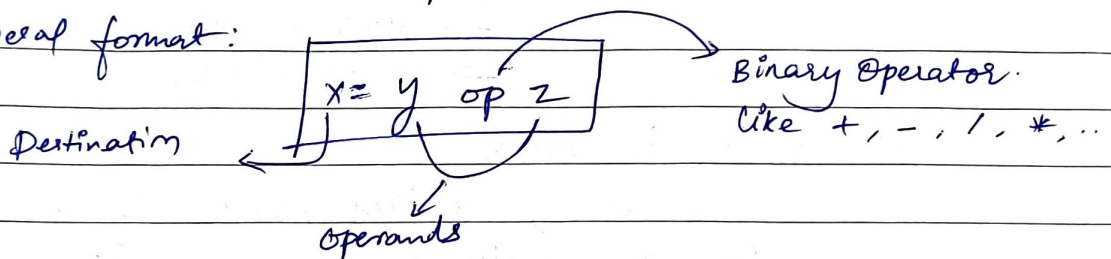
⇒ Three Address Code:

Power: Computational Power
 $PDA < LBA < \text{Turing Machine}$

* It is intermediate representation (IR) used in compilers. It breaks down complex operations into simple instructions, where each instruction contains almost three addresses (operands). They are

- * One result (usually a variable / temp variable)
- * Two source operands.

General format:



* Highly useful due to its simplicity and also as it helps in generating machine code or assembly later in the compilation process.

Example → Convert given expression in TAC

$a + b * c$

Solution:

$t_1 = b * c$

$t_2 = a + t_1$

⇒ Post-Machines on Post-Turing-Machine:

- * Power is as equivalent as Turing Machine.
- * More simpler tape-based model than TM.
- * It has only two symbols - "1" and "blank".
- * Instructions are very simpler too:

move, mark, or test.

* It operates by replacing strings, based on fixed rules.

Q

Given:

start string 'a'

rules:

$a \rightarrow ab$
 $b \rightarrow c$

DATE

--	--	--	--	--	--	--

sol

Derivation:

→

(start) - a

- a

$a \rightarrow ab$

- (ab)

$b \rightarrow c$

- a(c)

$a \rightarrow ab$

- (ab)c

$b \rightarrow c$

-

a.c.c ⇒ Final string //



must ~~not~~ ~~more~~

Learn more concepts

refer syllabus pdf.

~~penetration~~

Module - 5

⇒ Chomsky Hierarchy of Languages: (VTPS)

Type-0 - Unrestricted - Most Power - Handled By TM.

Type-1 - Context-sensitive - Needs LBA

Type-2 - Context-Free - Used For Programming Languages, Handled By PPA.

Type-3 - Regular - Very Simple, Handled by DFA/NFA.

⇒ Undecidability:

A problem is undecidable if there is no way to write a program that always gives the correct ~~input~~ answer for every input.
eg: we can't predict if a program will stop or run forever.

⇒ Recursive & Non Recursive Languages:

Recursive Language

You can decide if a string belongs or not (decidable)

Recursive Enumerable

You can recognize the string if it belongs, but may not halt if it doesn't.

Non RE:-

You cannot even recognize the strings in the language.

Problems Involved:

⇒ Code Generation :

DATE

--	--

--	--

--	--	--	--

Final step of compiler - that turns the code into machine level instructions (like Assembly).

Example: $x = a + b$ becomes Add x, a, b

Problems Involved - Undecidability problem, Code Generation Problem.

④ Must learn more (refer syllabus pdf).
